

Sri Lanka Institute of Information Technology



IT3012 : Intelligent Agents – Lab 3

Y3.S1.WD.AI.01.01

Malabe Campus

Student ID -	IT24100140
Student Name -	Prathiban HTP
Module Name -	Intelligent Agents
Module Number -	IT3012
LabSheet Number -	3
LabSheet Date -	2026/Aug/5

Task 1

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:
`'grid_size': (self.width, self.height)`
`'walls': list(self.walls)`
`'all_food': list(self.food_positions)`



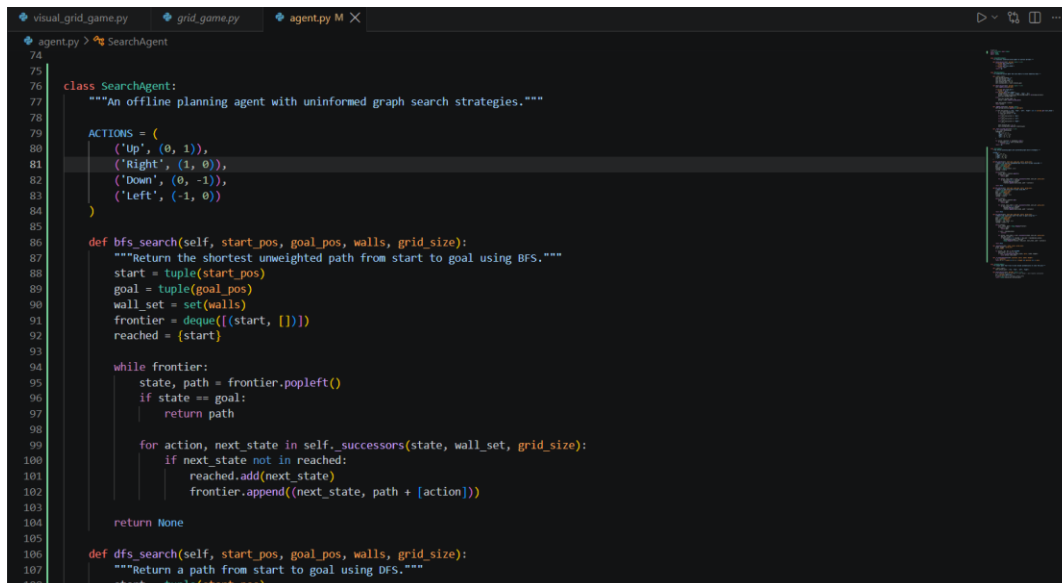
```
visual_grid_game.py M X  grid_game.py  agent.py
visual_grid_game.py > GridGameGUI
6  class VisualGridHuntGame:
43  def get_percept(self) -> dict:
52      wall Ahead = (
53          ahead_pos[0] < 0 or ahead_pos[0] >= self.width or
54          ahead_pos[1] < 0 or ahead_pos[1] >= self.height or
55          ahead_pos in self.walls
56      )
57
58      return {
59          'wall Ahead': wall Ahead,
60          'food Here': tuple(self.agent_pos) in self.food_positions,
61          'grid_size': (self.width, self.height),
62          'walls': list(self.walls),
63          'all_food': list(self.food_positions)
64      }
65
66  def execute_action(self, action: str):
67      self.steps += 1
68      new_pos = list(self.agent_pos)
69
```

Task 2

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS**: Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS**: Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS**: Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.



```
agent.py > SearchAgent
74
75
76 class SearchAgent:
77     """An offline planning agent with uninformed graph search strategies."""
78
79     ACTIONS = (
80         ('Up', (0, 1)),
81         ('Right', (1, 0)),
82         ('Down', (0, -1)),
83         ('Left', (-1, 0))
84     )
85
86     def bfs_search(self, start_pos, goal_pos, walls, grid_size):
87         """Return the shortest unweighted path from start to goal using BFS."""
88         start = tuple(start_pos)
89         goal = tuple(goal_pos)
90         wall_set = set(walls)
91         frontier = deque([(start, [])])
92         reached = {start}
93
94         while frontier:
95             state, path = frontier.popleft()
96             if state == goal:
97                 return path
98
99             for action, next_state in self.successors(state, wall_set, grid_size):
100                 if next_state not in reached:
101                     reached.add(next_state)
102                     frontier.append((next_state, path + [action]))
103
104         return None
105
106     def dfs_search(self, start_pos, goal_pos, walls, grid_size):
107         """Return a path from start to goal using DFS."""
108         start = tuple(start_pos)
109         goal = tuple(goal_pos)
110         wall_set = set(walls)
111         frontier = [start]
112         reached = {start}
113
114         while frontier:
115             state, path = frontier.pop()
116             if state == goal:
117                 return path
118
119             for action, next_state in self.successors(state, wall_set, grid_size):
120                 if next_state not in reached:
121                     reached.add(next_state)
122                     frontier.append((next_state, path + [action]))
123
124         return None
125
126     def ucs_search(self, start_pos, goal_pos, walls, grid_size):
127         """Return a path from start to goal using UCS (Uniform Cost Search)."""
128         start = tuple(start_pos)
129         goal = tuple(goal_pos)
130         wall_set = set(walls)
131         frontier = [(start, [], 0)]
132         reached = {start}
133
134         while frontier:
135             state, path, cost = heapq.heappop(frontier)
136             if state == goal:
137                 return path
138
139             for action, next_state in self.successors(state, wall_set, grid_size):
140                 if next_state not in reached:
141                     reached.add(next_state)
142                     new_cost = cost + 1
143                     heapq.heappush(frontier, (next_state, path + [action], new_cost))
144
145         return None
```

Task 3

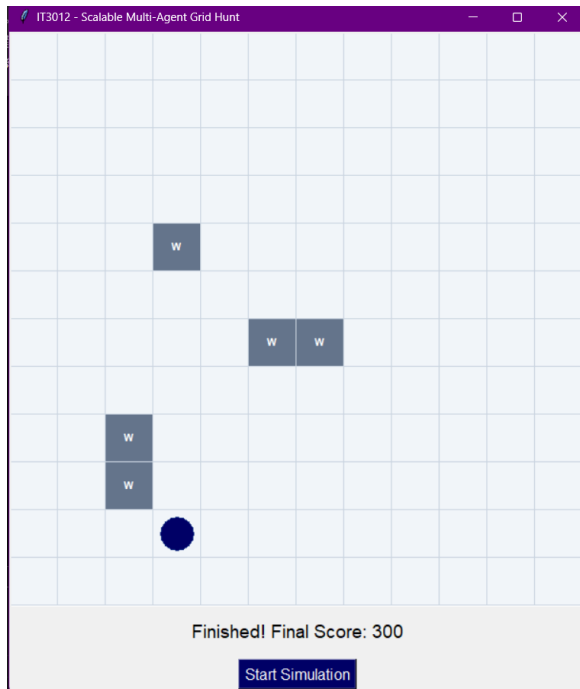
Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

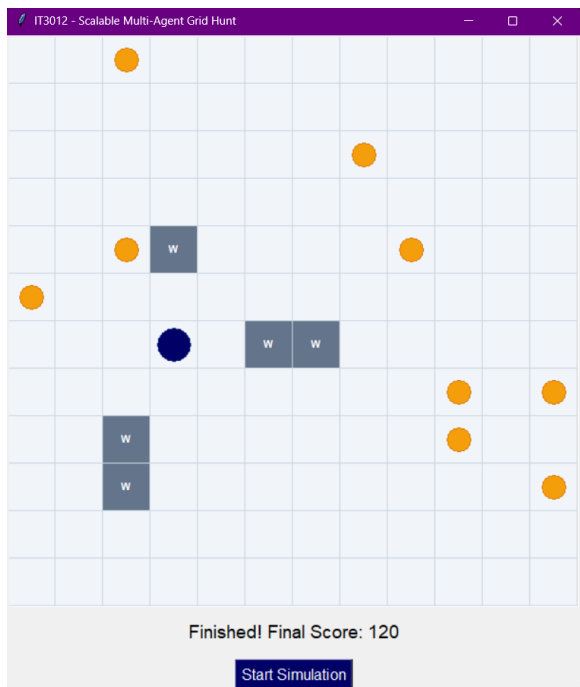
1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

```
visual_grid_game.py M  grid_game.py  agent.py M X
agent.py > SearchAgent
76 class SearchAgent:
77     ,
78
79
80
81
82
83
84
85
86     def __init__(self):
87         self.plan = []
88         self.active_algo = 'BFS'
89
90     def sense_and_act(self, percept: dict) -> str:
91         if percept.get('food_here'):
92             self.plan = []
93             return 'Suck'
94
95         if not self.plan:
96             start_pos = tuple(percept['agent_pos'])
97             food_positions = [tuple(food) for food in percept.get('all_food', [])]
98             walls = percept.get('walls', [])
99             grid_size = percept['grid_size']
100             self.plan = self._plan_to_closest_food(start_pos, food_positions, walls, grid_size)
101
102         if self.plan:
103             return self.plan.pop(0)
104         return 'Up'
105
```

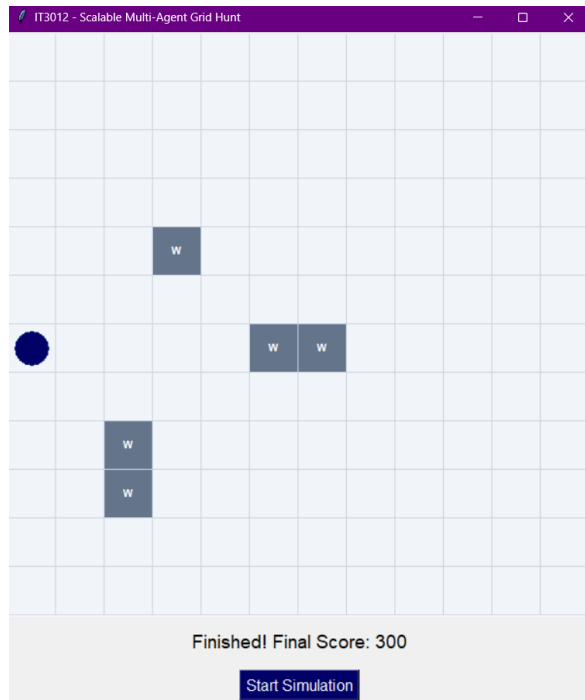
With BFS –



With DFS –



With UCS –



Part 2 Theoretical

1.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

State Space: The complete set of all possible valid states/positions the agent can be in within the environment.

Search Tree: The structure generated by the search algorithm while exploring those states from the initial state.

The same state can appear multiple times in a search tree if it can be reached through different paths, while it represents only one state in the state space.

2.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set keeps track of states that have already been visited.

It prevents the agent from: Revisiting the same grid position repeatedly. Creating cycles or infinite loops. Wasting time exploring the same states again.

Without it, DFS could repeatedly move through the same cycle of positions and may never reach the food, especially because DFS explores deeply before trying alternative branches.

3.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS explores nodes level by level, meaning it checks all positions one step away, then two steps away, and so on. Therefore, the first path it finds to the food contains the minimum number of moves, making BFS optimal in this grid.

DFS explores one path as deeply as possible first. It may travel far away from the food before backtracking, causing the winding and longer paths seen in the simulation

4.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

- BFS has a major memory bottleneck because its frontier can grow exponentially with depth.
- DFS uses much less memory, but it may explore very deep or poor paths and does not guarantee the shortest solution.

So DFS is more memory-efficient, while BFS provides better path quality in this equal-cost grid.